Frame Buffers

Computer Graphics
Instructor: Sungkil Lee

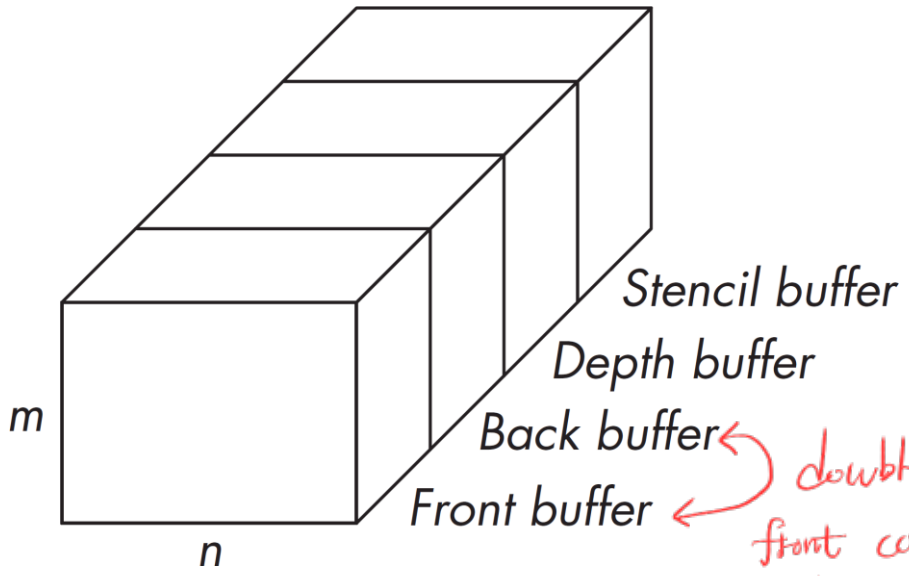
Today

- **Frame buffers**
- **Post-fragment operations and compositing**
  - Depth buffering
  - Blending
  - Compositing
- **Frame Buffer Objects (FBOs)**
  - Frame buffer objects (FBOs)
  - Render-to-Texture
  - Example Applications

Frame Buffers

Frame Buffers

- **Frame buffers store fragments generated via rendering pipeline in large 2D memory areas.**
  - Standard buffers: color buffers and depth buffers
    - Front and back color buffers are defined for double buffering.
  - Additional buffers: stencil buffers
    - In many cases, depth-stencil buffers are used together.



double buffering으로
front color buffer와
back color buffer를 바꿔준다.

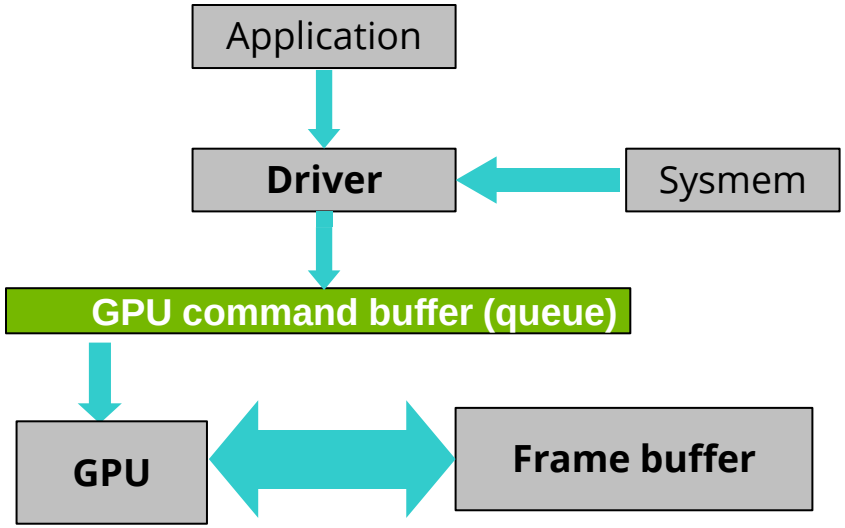
Frame Buffers

- **Frame buffers are defined by resolution and bit depth.**
  - Typical bit depths: 8 bpp, 24 bpp, and 32 bpp
  - This specification is similar to those of textures.
  - Hence, textures can be used as frame buffers, and we call such textures as render targets.

- **In practical scenarios, color buffers are solely important.**
  - While color/depth buffers are still actively used, stencil buffers are not that common in modern-style GL.
  - Even reading depth buffers is rare nowadays, because we can write our own depth outputs to the color buffers (e.g., RGBZ format).

Reading from Frame Buffers

- **Read operations**
  - While write operations are simply performed by draw calls by default, read operations can be called but take much longer than write.
  - This results from asynchronous threading model of OpenGL.
  - You can correctly read frame buffers after executing all the pending GPU commands in the GPU command queue.



Flow from application to GPU

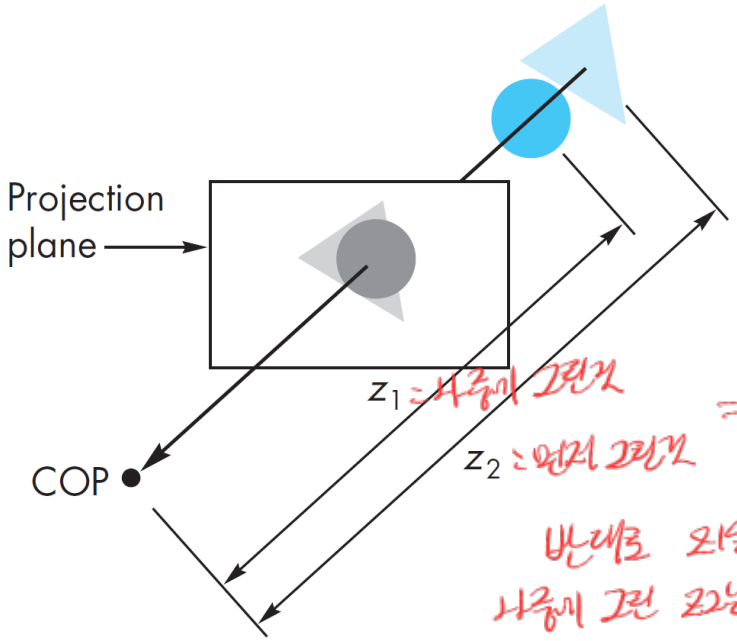
Post-Fragment Operations and Compositing

Post-Fragment Operations

- **Depth buffering**
  - maintains only nearest fragments in the buffer.
- **Linear blending**
  - alpha blending: use alpha channel
- **Logical operations: bitwise AND, OR, XOR, ...**
  - In this case, integer-format frame buffers should be used.
  - Advanced operations such as on-the-fly voxelization often use this.

Depth Buffering

- **Depth test/buffering (Z-test or Z-buffering)**
  - This is the most common operation in the raster graphics.
  - When enabling depth test, a fragment's depth is compared against the depth in the frame buffer.
  - When the fragment depth is smaller than the existing depth, the fragment is written to the color buffer with its depth value to the depth buffer.
  - Otherwise, the fragment is discarded.

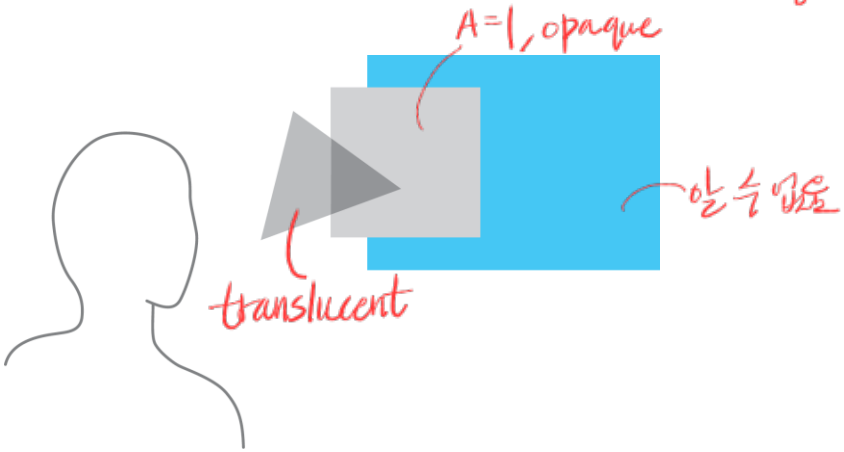


Blending in Frame Buffers

- **Alpha blending**
  - Linearly combines the fragment output (with its alpha value) and the colors in the frame buffer (with its alpha value).
  - Source: fragment output values (S_{rgb}, S_a)
  - Destination: framebuffer values (D_{rgb}, D_a)
- **Blending is specified in the host application.**
  - Framebuffer values are not visible in fragment shaders.
  - So, we tell OpenGL how to blend the fragment and the frame buffer values in the host application.
  - The allowed operations are mostly linear blending, and we can specify only a limited set of the linear blending operations.

Opacity and Transparency

- **Alpha values (the last 'A' component in RGBA) now used**
  - "A" (alpha) means opacity.
  - Translucency = 1 - opacity (or alpha)
- **General idea on transparency**
  - Opaque surfaces (A=1) permit no light to pass through.
  - Transparent surface (A=0) permit all lights to pass.
  - Translucent surface (0<A<1) permit pass some light.

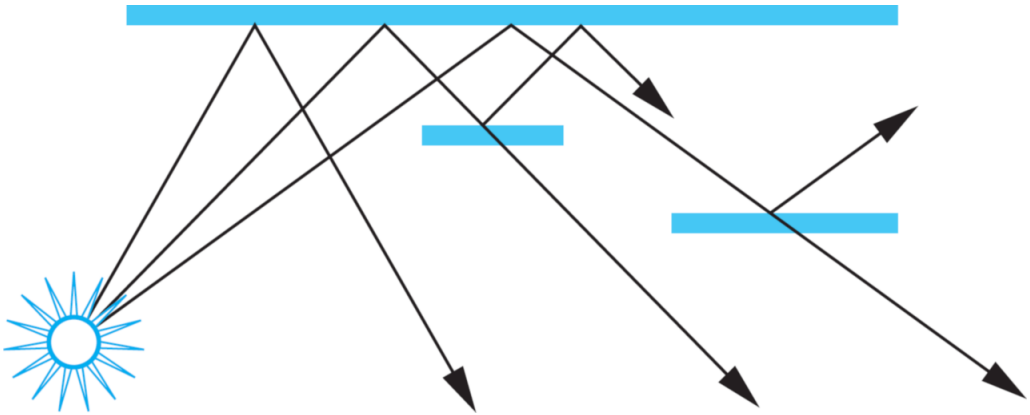


Translucent and opaque polygons

Translucency: Physical Models

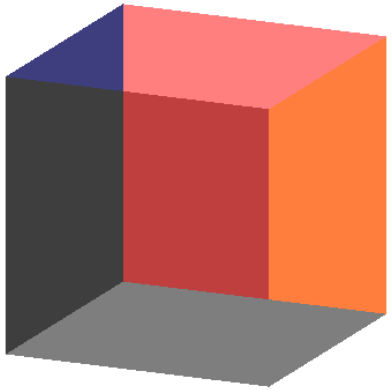
depth buffering에서는 모든 object가 opaque하다고 가정해서 순서 신경X

- **Translucency is a challenge in rendering.**
  - Dealing with translucency in a physically correct manner is difficult due to the complexity of the internal interactions of light and materials.
  - In particular, the order of rendering matters.

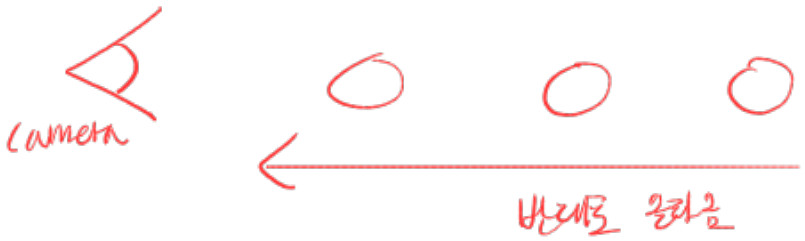


Scene with translucent objects

Order Dependency



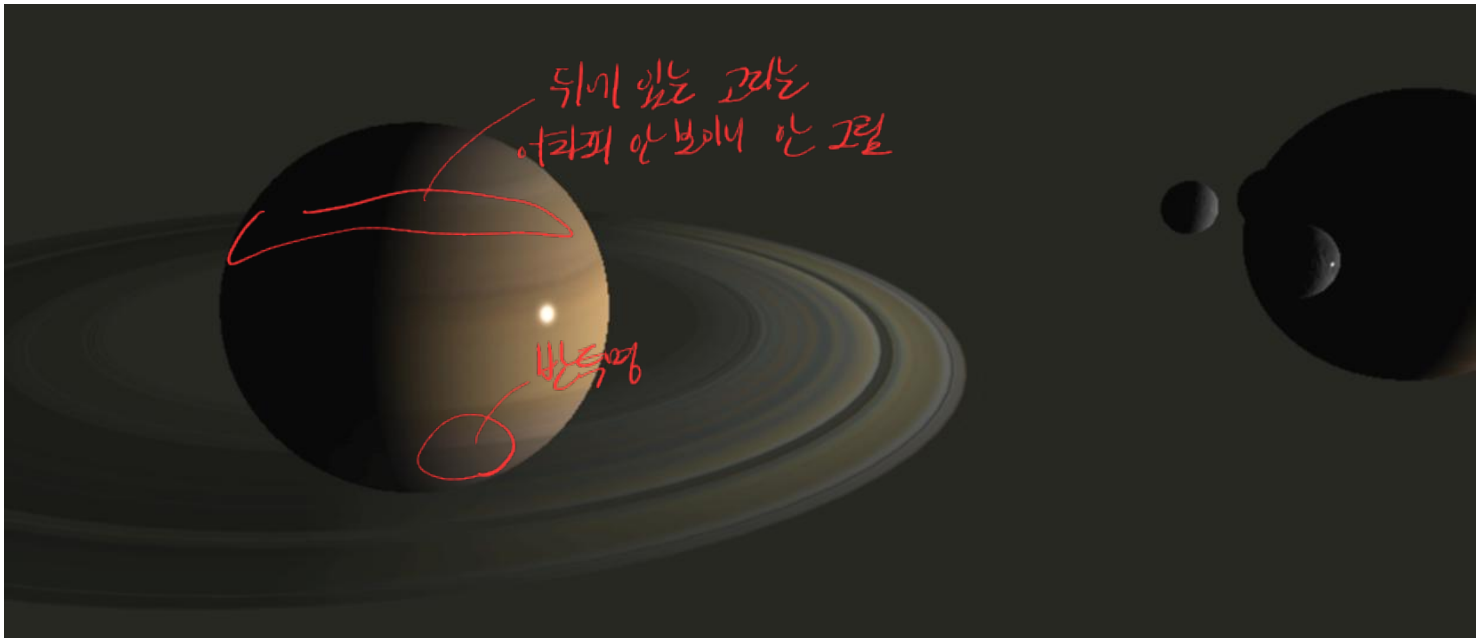
- **Is this image correct? Probably not…**
  - Recall that polygons are rendered in the order the application generated.
  - However, blending functions are order-dependent.
  - Order-independent transparency is still an on-going research topic.
- **Back-to-front blending**
  - We need to respect the spatial orders of objects.
  - Translucent objects are sorted, and then, are blended (or rendered) in a back-to-front order.



Blending for Transparency

- **In practice, few objects can be approximated.**
  - We first render opaque objects into the frame buffer, and then
  - render translucent objects and blend them with the frame buffer.

- **Example: the ring system of the Saturn**
  - Blend/render translucent rings in a back-to-front order.



Blending for Transparency

- **Simplest alpha blending**

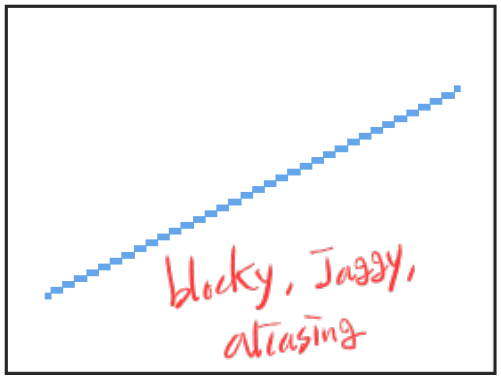
$$D'_{rgb} = S_{rgb} * S_a + D_{rgb} * (1 - S_a)$$
$$D'_a = S_a * S_a + D_a * (1 - S_a)$$

- **OpenGL example**

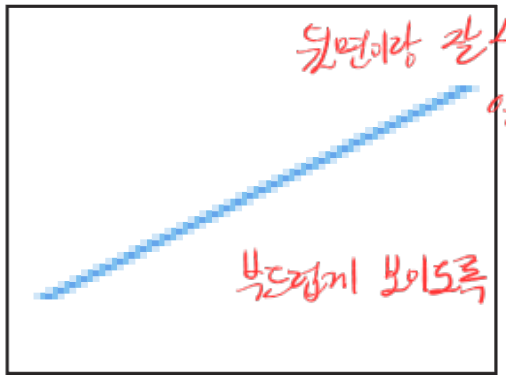
```
glEnable( GL_BLEND );
glBlendFunc( GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA );
...
```

Blending for Transparency

- **Another example with antialiasing**
  - Point/line/polygon are smoothed for antialiasing.
  - The coverage factors to neighboring pixels are computed, and alpha-blended.



original jaggy line



anti-aliased line

  - OpenGL example:

```
glEnable( GL_LINE_SMOOTH );
glEnable( GL_BLEND );
glBlendFunc( GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA );
```

Additive Blending

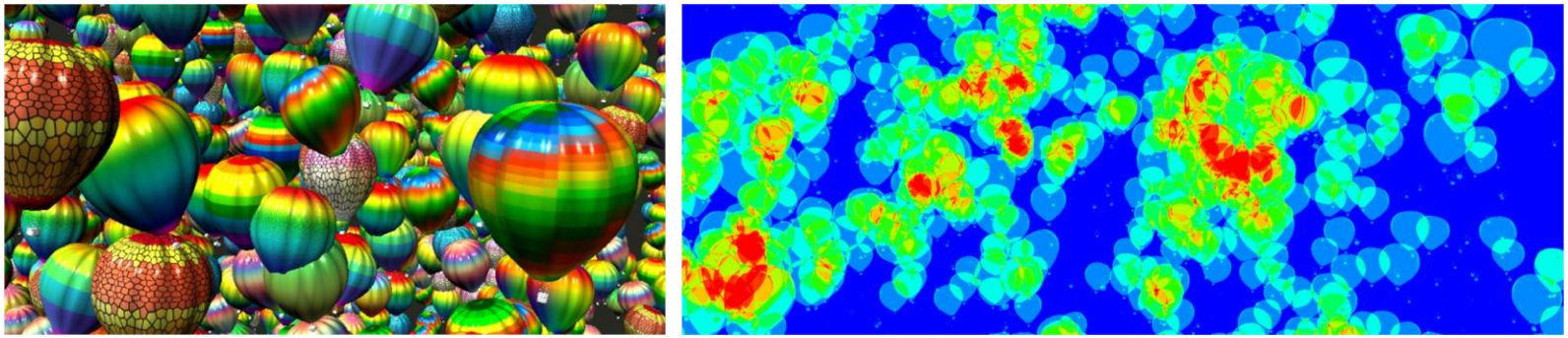
- **Additive/accumulative blending**

frame buffer를 그대로 두고 source에서 주어진 alpha를 weighted한 color를 더해줌 — 이렇게 계속 더해줌

$$D'_{rgb} = S_{rgb} * S_a + D_{rgb} * 1$$
$$D'_a = S_a * S_a + D_a * 1$$

- **Example**
  - Counting of the number of drawn fragments

얼마나 depth에서 겹쳐진 fragment 수를 count



Color-mapped number of incoming fragments

Compositing

- **Compositing**
  - When you are blending pre-rendered textures, you can easily implement blending in your shaders.
  - This is called the compositing rather than blending.
  - Compositing is common in many UI/web rendering engines.
  - Try to distinguish the compositing from blending.

- **Examples of the compositing**
  - You have background textures.
  - You have foreground objects with alpha (e.g., leaves of a tree)
  - You can composite the background and foreground objects using alpha blending equations, while disabling blending.

Frame Buffer Objects (FBOs)

display와 연결되지 않은 frame buffer

Frame Buffer Objects (FBOs)

- **Textures can be used as frame buffers,**
  - because they reside inside the physically same memory areas.

- **We perform the same renderings, yet with our own frame buffer (objects).**
  - We need to specify additional frame buffers whose render targets are attached to textures to replace the default frame buffers.
  - Such frame buffers are called the frame buffer objects.

Render-to-Texture (RTT)

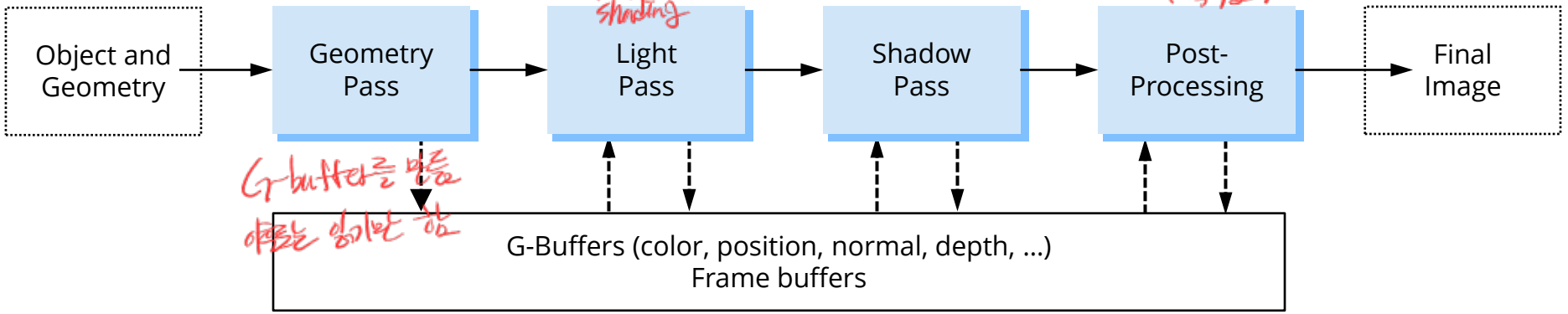
- **Frame buffer objects allow us to perform "render-to-texture."**
  - The rendering results are now stored in the texture memory.
  - We may read them back into main memory for further uses.
  - We may use them as if a static texture in shaders.
  - RTT is a primitive for many postprocessing.

- **Example: defocus blur**
  - Pass 1: render the objects' RGB colors and depths
  - Pass 2: apply a box blur by the distance to the focal plane depth.

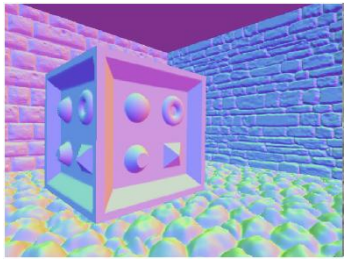
Render-to-Texture: Example Applications

Deferred Rendering

- **General idea**
  - Multi-pass approach yet with a single geometry pass
  - Render attributes of the geometry and its material first on geometry pass.
    - The buffer is called the *G-buffer*.
  - Perform shading and postprocessing later after the geometry pass



Color



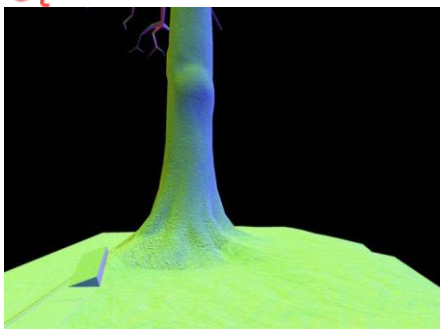
Normal

Depth

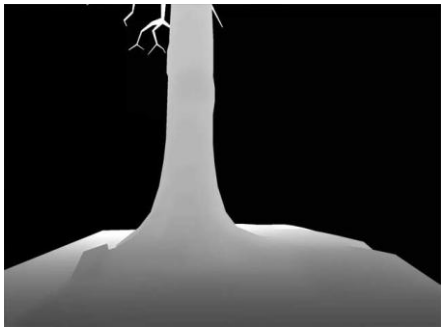
[Thaler 2012]

Examples

Deferred Shading, Shawn Hargreaves, GDC 2004



normals



depth

specular intensity

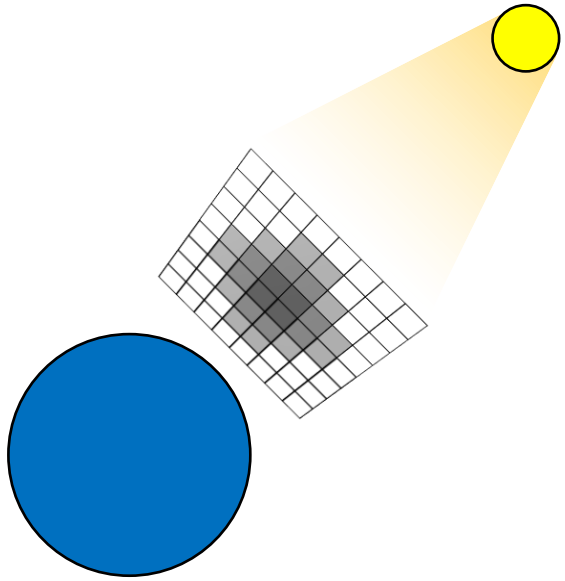
specular power

Diffuse color (albedo)

Deferred lighting result

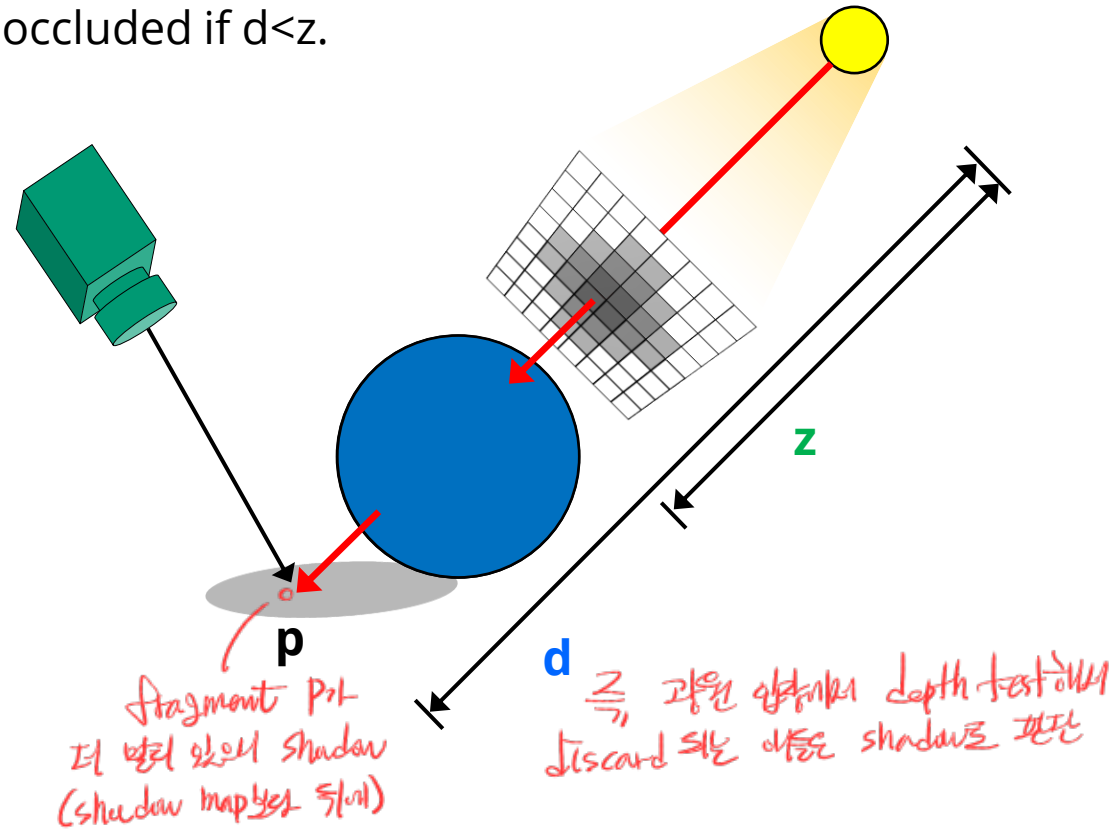
Shadow Mapping

- **First pass: shadow map rendering**
  - Render the depth image (shadow map) from the light viewpoint.



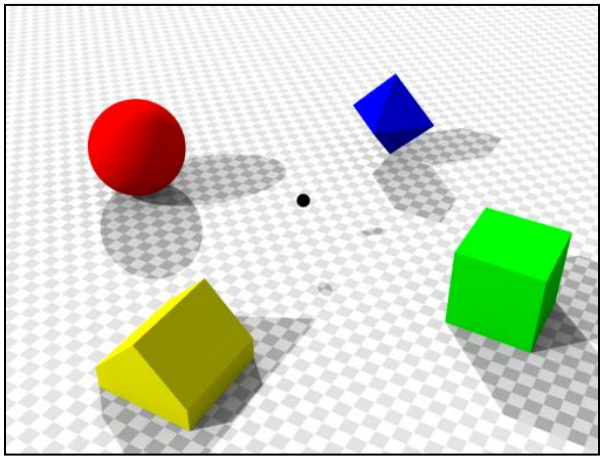
Shadow Mapping

- **Second pass: calculating shadow factors**
  - Calculate shadow factors for each fragment that determines whether the fragment is shaded.
    - Compare depth values of the current fragment (z) and the shadow map (d).
    - Point p is lit (no shadows) if d>=z.
    - Point p is occluded if d<z.

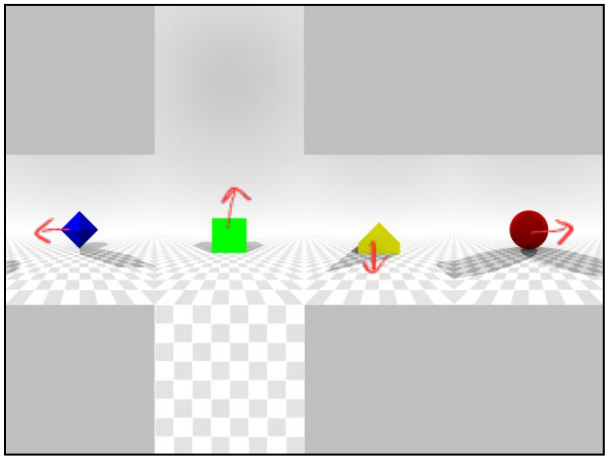


Online Cube-Map Rendering

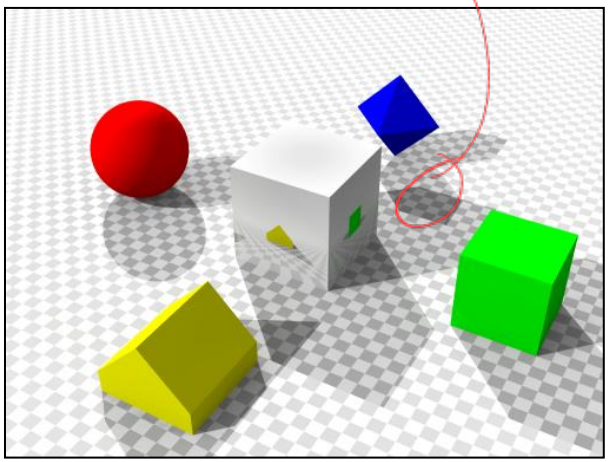
- **Cube mapping:**
  - Environment mapping via a 6-face cube map
  - Could be a static texture or dynamically-generated for every frame



place a viewer in a scene



render 6-face cube map



apply texture to the surface